CSC4026Z Chat Server Documentation

Encryption

The encryption used by the chat server is a **modified** version of the Wireguard protocol. Wireguard is a modern VPN solution with a small and tractable cryptographic component. Note that our chat server uses a modified Wireguard, see below for the parts you need to implement and the parts you must leave out.

Wireguard's justification and thinking can be explored in the definitional paper.

The full protocol is specified here, though remember we won't use all of it. Wireguard was designed to carry encapsulated IP packets; in this prac we will use part of Wireguard's standard to simply encapsulates chat messages.

Your identity and the server's identity

Each student has been assigned their own static key which serves as your identity when connecting to the Wireguard port. You can retrieve your personal static keypair by browsing to https://csc4026z.link/keys, then entering your surname and student number.

The Server's static *public* key is (in a Python form):

```
SERVER_STATIC_PUBLIC_KEY=b'f,^\xc0Cb\xf3\x937\xbf\x11\x14"\xed\x13\x0b
```

Important information when implementing your encrypted chat client

1. Before you can send chat messages, you must perform a simplified Wireguard handshake with the server.
2. The Wireguard specification talks about initiators and responders. For this practical, the initiator is always your chat client, and the responder is always the chat server.
3. The Wireguard specification includes support for rekeying,

timers and more. Our simplified Wireguard implementation ignores them; you don't need to rekey, send keepalives, or rotate your keys.
4. There are four keypairs involved in various ways in each chat session. Your client will have two, a static and an ephemeral keypair, and the server also has two, a static and an ephemeral keypair.
5. The static keypairs are the long-lived identities of each side. Every student is assigned a unique static keypair. The server only allows a single connection per student.

The CSC4026Z Wireguard protocol

From the the paper you will implement these Sections for the basic marks:

- Section 5.4.2: Sending an initiation handshake
- Section 5.4.3: Handling the handshake response
- Section 5.4.4: Cookie MACS — Only calculate `mac1`, don't calculate `mac2`
- Section 5.4.5: Key derivation
- Section 5.4.6: Transport data messages (but you don't need to pad the data before encryption)

Excluded parts

You explicitly **MUST NOT** implement these parts of the spec for the basic marks:

- Section 5.2 (Pre-shared symmetric keys are not used.)
- Section 5.3 (DoS mitigation with `mac2`)
- Section 5.4.4: Cookie MACS — Don't calculate `mac2`
- Section 5.4.7 (Cookie replies)
- Section 6 in its entirety (timers, key rotation, rekeying, keepalives, cookies, etc)

Top marks

5% of the project mark is allocated towards implementing an extra feature of the Wireguard protocol. Only implement this once you have a fully working simplified Wireguard implementation.

Library usage

The Wireguard paper relies on several functions in its protocol description, such as `DH()`, `DH-Generate()`, `AEAD()` and so on. You can leverage cryptographic libraries to implement the underlying cryptography (i.e. hashing functions, key generation, elliptic curves, and authenticated encryption).

For example, `DH()` and `DH-Generate()` can be implemented with the NaCL library as:

```
import nacl.bindings
import nacl.public
def DH(private_key, public_key):
    return nacl.bindings.crypto_scalarmult(n=private_key, p=public_key

def DH_Generate():
    private_key = nacl.public.PrivateKey.generate()
    return (private_key, private_key.public_key)
```

Additional libraries are cryptography, and the built-in hashlib.

We strongly recommend implementing the functions listed in Section 5.4.4 of the Wireguard page first before trying to tackle the modified Wireguard protocol. Note that the `XAEAD()` does NOT need to be implemented (you're implementing a simplified spec, not the official spec). In the next section we provide sample inputs and outputs for each function to check that you are implementing them correctly.

Sample Functions

All the samples below show valid inputs and the expected outputs for the functions. You will need to implement the functions yourself. The examples below were performed in a Python REPL.

`DH_Generate()`

```
>>> (private_key, public_key) = DH_Generate()
>>> print(private_key)
b'\xb0)e\xdbZ\x01\x8f\x0f\xf5\x91\x88<\xab\x15\x14\x95\xb3\x92\xbd&3\x
>>> print(public_key)
b'\x14\xde\xd1\x90m?\x0eaBa\xbb\xf8\\\x08\xdd\xfd\x08\xa7?^\x9f\xcb\x1
```

If this is working, it'll always return a new keypair.

DH()

```
>>> private_key = b'\xb0)e\xdbZ\x01\x8f\x0f\xf5\x91\x88<\xab\x15\x14\x
>>> public_key = b'\x14\xde\xd1\x90m?\x0eaBa\xbb\xf8\\\x08\xdd\xfd\x08
>>> print(DH(private_key, public_key))
b'p\x06\xe4\x7f\xce\x87\x88\xe2\xb9\xd1\xb0\xb7\xf3\x0e}\xb1\xc6{g\xae
```

Fundamentally, `DH()` is the core of most modern cryptographic systems. Each party in a communication has a public and a private key which are related. They share their public keys with each other. The "magic" of secure communication boils down to the fact that:

```
# On the client I know the server's public key and my private key so I
>>> server_public_key = b'f,^\xc0Cb\xf3\x937\xbf\x11\x14"\xed\x13\x0b\
>>> client_private_key = b'\xb0)e\xdbZ\x01\x8f\x0f\xf5\x91\x88<\xab\x1
>>> print(DH(client_private_key, server_public_key))
b'\x1a\x7f=\xf5w\xcd\xfe\xdb\xdd\xf1\xbd\xcd$\xa4\xc3\xbclLk*\xbf\x0ec

# On the server, it knows the client's public key and the server's pri
>>> server_private_key = ... # redacted
>>> client_public_key = b'\x14\xde\xd1\x90m?\x0eaBa\xbb\xf8\\\x08\xdd\
>>> print(DH(server_private_key, client_public_key))
b'\x1a\x7f=\xf5w\xcd\xfe\xdb\xdd\xf1\xbd\xcd$\xa4\xc3\xbclLk*\xbf\x0ec
```

When the client and server both run `DH()`, they wind up with the identical output in spite of the fact that they never had to exchange that value. The output of `DH()` becomes the central part of the secret used to communicate.

AEAD()

`AEAD()` needs both an `encrypt` and a `decrypt` version.

```
>>> key = b':\xb6\x90\xbd\n:\x18Z88"\xd8a\x08\x9f\xa7\x9c\xc7\xcb\x01\
>>> counter = b'\x00'*12 # Read the standard carefully for how this mu
>>> plaintext = b"attack at dawn"
>>> authtext = b'\x8e2\x89\xe2\x14\xfd\x16\x19o\x06\xc9\xb2\xd9\xe8F\x
>>>
>>> encrypted_data
b'\xfbv\x84\xea\xd0S\n\xc1\x16\x9et\xd5\xa4/\xeee\x9a\xa9MR\xe3\xd5p3\
>>> AEAD_decrypt(key, counter, encrypted_data, authtext)
b'attack at dawn'
```

Hash()

```
>>> Hash(b'Noise_IKpsk2_25519_ChaChaPoly_BLAKE2s')
```

```
b"`\xe2m\xae\xf3'\xef\xc0.\xc35\xe2\xa0%\xd2\xd0\x16\xebB\x06\xf8rw\xf
```

`MixHash()`

This function is not defined in the paper, but will be a useful convenience function because a number of operations require the hash of concatenated inputs. Use whenever you see `Hash(.. || ..)` in the paper.

```
>>> input1 = b'a'*50
>>> input2 = b'b'*50
>>> MixHash(input1, input2)
b'#B\x17\x17\xbe=\xfcc\xd6\xb5@81\t\x8dh\x88\x9b\xb3\xa8\xb9\xb2n\n\x0
```

`Mac()`

```
>>> key = b':\xb6\x90\xbd\n:\x18Z88"\xd8a\x08\x9f\xa7\x9c\xc7\xcb\x01\
>>> input = b'I am a message without a MAC, but only for now.'
>>> Mac(key, input)
b'*\xbd\x8ak4%\xe4\xb0\xe7\x96\xe5z\x14q\xdd!'
```

`Hmac()`

```
>>> key = b':\xb6\x90\xbd\n:\x18Z88"\xd8a\x08\x9f\xa7\x9c\xc7\xcb\x01\
>>> input = b'I am a message without an HMAC, but only for now.'
>>> Hmac(key, input)
b'\x1ew,:\x03\xdd\x0b\x1e\x96\n\x00J\x8c\xe1QzQ\xff\xb8\x02\xcb\xa29\x
```

`Kdf()` functions

There are three `Kdf()`-derived functions to create. Carefully read the specification for their construction.

```
>>> key = b':\xb6\x90\xbd\n:\x18Z88"\xd8a\x08\x9f\xa7\x9c\xc7\xcb\x01\
>>> input = b'Choose your LLM adventure folks.'
>>> Kdf1(key, input)
b'0fO\x0e\x0f\xb2\xf4\xaa\xcc\x14\x9c\x84\x8a\xb0D\xd3i\xa6\xac\xbf\xa
>>> Kdf2(key, input)
(b'0fO\x0e\x0f\xb2\xf4\xaa\xcc\x14\x9c\x84\x8a\xb0D\xd3i\xa6\xac\xbf\x
>>> Kdf3(key, input)
(b'0fO\x0e\x0f\xb2\xf4\xaa\xcc\x14\x9c\x84\x8a\xb0D\xd3i\xa6\xac\xbf\x
```

`Timestamp()`

```
>>> t = 1744366282.5143921
>>> Timestamp(t)
b'@\x00\x00\x00g\xf8\xea\xd4\x00\x07\xd9X'
```

In reality your `Timestamp()` function won't take a parameter, it'll use the current time; the above `t` parameter is so you can test the output.

> **NOTE:**
>
> The server's clock is automatically kept up-to-date. If your own computer's time source is more than 10 seconds off from the server, it will ignore your handshake request. Make sure your computer is configured to automatically update its time.

Wireguard Protocol implementation notes

The paper explains clearly how to create the initiation message. Through that process you will track both a hash value and a chaining key value. In the worked examples below, we list the functions from the paper, then show what the concrete values are after those steps were run. The examples rely on these static keys:

```
client_private_key = b'\x99x\x93eP\xdd\xb7h\xd5dJ\xc7\xa5~\x83\xbdX\x0
server_public_key = b'f,^\xc0Cb\xf3\x937\xbf\x11\x14"\xed\x13\x0b\x9f\
```

> **NOTE:**
>
> Your own static key must be used when you attempt to connect to the server, the key above will not work against the live server.

Initiation message construction

Here is the worked example for constructing the initiation message, following the steps listed on page 11 of the paper (recall that `DH-Generate()` and `Timestamp()` values are fixed in this worked example, and will always return different results in reality):

```
# chain_key = Hash(Construction)
```

```
chain_key == b"`\xe2m\xae\xf3'\xef\xc0.\xc35\xe2\xa0%\xd2\xd0\x16\xebB

# hash = Hash(chain_key || Identifier)
hash == b'"\x11\xb3a\x08\x1a\xc5fi\x12C\xdbE\x8a\xd52-\x9clf"\x93\xe8\

# hash = Hash(hash || S_R_pub)
hash == b'1\xd6\xfb\xdb\xb5F\x13`z\x89\x8cu\xf7,\x8c5x\xa1\xae@#\xf3tu

# (E_I_priv, E_I_pub) = DH-Generate()
E_I_priv == b'\xac\x03\x18b0\xc4\xf7\xd4*\xa7-\x81&\xfb\xc7\xb3PG0\xae
E_I_pub == b"\xb1\x13\xb4\xd3\x00R'\x8b\x80\xd1\xcc\xc8X\x1bYf(4\xce&\

# chain_key = Kdf1(chain_key, E_I_pub)
chain_key == b'/U?\xfe\\"\xd6B%\xdb\x94a\xa5\x0b\xf5\x1e\x0f\xbd\r[l\x

# hash = Hash(hash || E_I_pub)
hash == b"*\xf8N;\xd3\x9c!\xc3x\x14'\x01\x8eHh\xac\x0c3\xf2\xb4\xd7#c\

# (chain_key, key1) = Kdf2(chain_key, DH(E_I_priv, S_R_pub))
chain_key == b'\xb9\xb7\xc6[$4\xc1\xb1\xb8O\xd5\xf42\xb4\xd8\xa25\xea\
key1 == b'S%mJ\xe2\xc4AS\xc4\xfcX\x88\x86\xa0\th\xf5\xfb5\xffO\x08}\xa

# msg_static = AEAD(key1, 0, S_I_pub, hash), an encryption operation
msg_static == b'\xc7\x12ry\x04\xb9\xc3\xaf\x9a\xf4\x7f \xf1\x98\xf3rA\

# hash = Hash(hash || msg_static)
hash == b'\x82\xc6E\xc2\xa3\xf4\xe83\x81\x99_\xf64kx-\xff\x18\x1e9XL:[

# (chain_key, key2) = Kdf2(chain_key, DH(S_I_priv, S_R_pub))
chain_key == b'\xe0\\UH\\\x12\x9a\xb4\xcc\xd0\r\xa9\xd2\xac\xc7\xb1]ky
key2 == b'az\x7f\xef\xa9V\x7f#\xa5\xa8\xf0s\xca\xaft\xe76\x19\xf9\xc7\

# timestamp = AEAD(key2, 0, Timestamp(), hash), an encryption operatio
msg_timestamp == b'\xc6?XF\x845JvXfm\xf9\xfa0\xf4\xb2\x18\xd6\xf9\x046
# NOTE: In the above example, Timestamp() returned the value 174437702

# hash = Hash(hash || msg_timestamp)
hash == b'r\xdb\rg\x14\xa2\xff\x13h\xf8K\x9dL\xec\x81\xbf\xa6Q\x15\xf3
```

From the above code, a Wireguard initiation packet (without the MAC fields) was created with these values:

```
type = b'\x01'
reserved = b'\x00'*3
sender = 4001697114 # Random number, fixed for this worked example
ephemeral = E_I_pub
static = msg_static
timestamp = msg_timestamp

The raw bytes for the message are:

msg == b"\x01\x00\x00\x00Z\r\x85\xee\xb1\x13\xb4\xd3\x00R'\x8b\x80\xd1
```

MAC construction

The last step in creating the initiation message is to calculate the `mac1`, following the steps listed on page 12 of the paper:

```
# Mac(Hash(Label-Mac1 || S_R_pub), msg)
mac1 == b'/\x1a\x0b\xedH\xbb\xe63\xe8?\xa2bvX\x84\xb3'
mac2 == b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00
```

Responder message consumption

After a well-formed initiation message is sent to the server, it responds with a message that you must parse and decrypt. For handling the server's response, keep in mind that the paper describes how to CONSTRUCT the message; to consume the message you should carefully note the keys being used when performing `DH()`. Here is a worked example to decrypt the responder's message, continuing with the `hash` and `chain_key` from the initiation message. It follows the steps listed on page 11 of the paper:

```
# The chain_key and hash are maintained through the initiation sequenc
chain_key == b'\xe0\\UH\\\x12\x9a\xb4\xcc\xd0\r\xa9\xd2\xac\xc7\xb1]ky
hash == b'r\xdb\rg\x14\xa2\xff\x13h\xf8K\x9dL\xec\x81\xbf\xa6Q\x15\xf3

# For our worked example, the server's sender index is 3811305028, and
# E_R_pub is b'\xe5\xd0!\x98z\x12\xb5\xf8&\x17\xfe\x14K\x9exe(\x1bK\xc

# chain_key = Kdf1(chain_key, E_R_pub)
chain_key == b'u\xaaKs7\x866N-\xfd8\xb4\x8f\x83\xa0\xaeR\xbcM=\x84\xc6

# msg_ephemeral = E_R_pub
msg_ephemeral == b'\xe5\xd0!\x98z\x12\xb5\xf8&\x17\xfe\x14K\x9exe(\x1b

# Hash(hash || msg_ephemeral)
hash == b".'H\xa8\xe6\xa0\xf7K\xff\xbe\xc3\x991\xbd\xd3$\x1b\xa6\xd9z\

# chain_key = Kdf1(chain_key, DH(E_I_priv, E_R_pub))
chain_key == b'M$O@\x8e\xc9\x17\x86Y\xf1\xc4\xa5#\xdb\x00\xf7\xf8\xf7B

# chain_key = Kdf1(chain_key, DH(S_I_priv, E_R_pub))
chain_key == b'Y\xbb\xe2\xa1\xfe3\x88E\x96\x86\xc0\xfa\x7f\x0f\xfb~s`f

# (chain_key, tmp, key3) = Kdf3(chain_key, Q)
chain_key == b"\x12\xa4t\x15\xf3\x1b\xda\xe7\x9b\xe1\xb7\xf7\xc2)\xc1'
tmp == b'\x96\xe6(R*\xe3\x7fv^\x99\xbc\xe9n\xc0\x08\xd83#.\xb9\xcdE\x0
key3 == b'\xbd\xa1d\tZS\x85\xe0\xf3;]+$\xb2\xe4Po\xae\xec\x14d\x1a\x98

# Hash(hash || tmp)
hash == b"G\xb8N\xb6h\x18\xa4\x9a2\x0f\n\xe1Z\x158\x99\xfazHg'\xf3\xb0
```

```
# msg_empty = AEAD(key3, 0, empty, hash), a decryption operation
msg_empty == b''

# Hash(hash || msg_empty)
hash == b'K\x03\x8fh\x1a\x19\x11\x8fu\xde\xd1\xb8\xf6\xef\xc1\xaa\xa0\
```

Transport Key derivation

The key derivation below follows the steps on page 12 of the paper:

```
# (T_I_send, T_I_recv) = Kdf2(chain_key, ϵ)
sending_key == b'\xd0\x98\xff\xa8\xf4u&\xc4$\x94\xcd&*X\xbfc\x91_\xc3l
receiving_key == b'\x032\xb2\xbe\xae"<\'}\x97\x08@w\x98\x10l\xb9\xd4|\

# (N_I_send, N_I_recv) = 0
N_I_send == 0
N_I_recv == 0
```

Transport message construction

At the beginning of our session, `N_I_send = 0`. The worked example for a transport message is below, following the steps on page 12 of the paper:

```
# P = P || 0^16... # We only send data, no padding
p == b'\x82\xacrequest_type\x01\xaerequest_handle\xce\xf4\x02\xe7\xd5'

# msg_counter = N_I_send
msg_counter == 0

# msg_packet = AEAD(T_I_send, N_I_send, P, ϵ) # This is an encryption
msg_packet ==  b"\xb8\xe3ps\xbaB\x17\xdaC9\x0f\xb9\xee\xbb\n\x1d\xe3\x
```

```
# N_I_send += 1
N_I_send == 1
```

Transport message consumption

There is no worked example for this, you should derive it yourself.
